

Project Deliverables

1. Source Code

The complete, working source code is hosted at:

Repository: <https://github.com/Narayana-murthy-chikkala/Signzy> (branch: main)

Live Deployment: <https://signzy-omega.vercel.app>

Tech Stack

LAYER	CHOICE
Frontend	React 19 (Vite) + Tailwind CSS + React Router + Recharts
Backend	Node.js + Express.js — MVC with the Strategy design pattern for routing
Database	MongoDB Atlas + Mongoose
Testing	Jest + Supertest (unit + integration against an in-memory MongoDB)
API Testing	Postman collection (server/postman_collection.json)

Project Structure

client/src/	
components/	Layout, tables, cards, modal, badges, toasts, progress bars
pages/	Dashboard, Vendors, RouteTester, Metrics, Logs, Health, AIRuleGenerator
services/	axios API client (tags every request with the active role)
hooks/	useMetrics, useLogs, useHealth
context/	AppContext (shared vendor list), ToastContext, RoleContext (Admin / Client persona switch)
utils/	formatting + per-capability example payloads
server/	
config/	DB connection, runtime constants
controllers/	vendor, routing, metrics, logs, health, ai-rule, routing-config, advice
models/	Vendor, RoutingLog, RoutingConfig (Mongoose schemas)
routes/	Express routers
middleware/	centralized error handler (normalizes Mongoose errors to clean 4xx)
services/	vendorSimulator, metricsService, loggingService, routingEngine, strategyAdvisor, fallbackAdvisor
strategies/	priority / weighted / roundRobin / lowestLatency / lowestCost / healthBased / failover / featureBased (Strategy pattern, 1 file each)
utils/	filterVendors, ratelimiter, computeAvailability, vendorCache, aiRuleGenerator, capabilityResponses, seed
tests/	Jest + Supertest suite

2. README

The full README (setup instructions, API reference, routing strategy details) lives at the repository root: `README.md`. Summary below.

Problem

Modern platforms depend on multiple third-party vendors for the same capability (KYC/PAN verification, OCR, SMS, payment processing, document validation) — each differing in cost, latency, success rate, rate limits, error rate, availability, and supported features. This project exposes one unified API that decides which vendor actually handles a given request, so the client never needs to know which vendor was used.

Setup

```
npm run install:all    # installs server + client dependencies
npm run seed          # seeds 5 sample vendors into MongoDB
npm run dev            # runs backend (:5000) and frontend (:5173) together

cd server && npm test  # runs the automated test suite (in-memory Mongo)
```

Mandatory APIs (all implemented)

METHOD	ROUTE	PURPOSE
POST	/vendors	Register a vendor
GET	/vendors	List vendors
PUT	/vendors/:id	Update a vendor
DELETE	/vendors/:id	Delete a vendor
POST	/route	Route a request to the best vendor
GET	/vendor-metrics	Per-vendor + aggregate metrics
GET	/routing-logs	Paginated/filterable routing decision logs
GET	/health	Server + DB + vendor health snapshot

Bonus endpoints: /ai-rule-generator, /routing-configs, /strategy-recommendation, /fallback-suggestions.

Access model

Every request to /route is tagged with an x-role header (admin or client) by the dashboard's persona switch. admin gets the full response, including which vendor handled the request and why. client gets the identical envelope with vendorUsed, routingReason, and failoverHistory stripped — enforced server-side, not just hidden in the UI, so the "client never knows which vendor was used" requirement actually holds at the API boundary. See §4 for a live example of both.

3. Sample Vendor Configs

The seed script (server/utils/seed.js) populates 5 vendors covering all 5 capabilities, deliberately including one unhealthy and one down vendor so filtering/failover is visible immediately. This is the actual current state of the database, pulled live via GET /vendors:

NAME	PRIORITY	WEIGHT	COST	TIMEOUT	RATE LIMIT	HEALTH	UP?
Vendor A	5	5	\$0.10	2000ms	100/min	healthy	yes
Vendor B	3	3	\$0.05	1500ms	50/min	healthy	yes
Vendor C	4	4	\$0.20	3000ms	200/min	healthy	yes
Vendor D	2	2	\$0.02	1000ms	30/min	unhealthy	yes
Vendor E	1	1	\$0.15	2500ms	100/min	healthy	no (down)

Capability coverage

VENDOR	SUPPORTED CAPABILITIES
Vendor A	PAN_VERIFICATION, OCR, SMS
Vendor B	PAN_VERIFICATION, PAYMENT_PROCESSING, DOCUMENT_VALIDATION
Vendor C	OCR, SMS, PAYMENT_PROCESSING, DOCUMENT_VALIDATION

VENDOR	SUPPORTED CAPABILITIES
Vendor D	PAN_VERIFICATION, DOCUMENT_VALIDATION
Vendor E	OCR, SMS, PAYMENT_PROCESSING

Raw config (as sent to POST /vendors)

```
{
  "name": "Vendor A",
  "priority": 5,
  "weight": 5,
  "costPerRequest": 0.1,
  "timeoutMs": 2000,
  "rateLimitPerMinute": 100,
  "supportedFeatures": ["PAN_VERIFICATION", "OCR", "SMS"],
  "healthStatus": "healthy",
  "isActive": true
}
```

4. Sample API Requests & Responses

Every capability gets a distinct, realistic request/response shape — captured live from the running application immediately after a fresh seed (real request IDs, real simulated latencies).

PAN Verification

REQUEST

```
POST /route
{
  "capability": "PAN_VERIFICATION",
  "payload": {
    "pan": "ABCDE1234F",
    "name": "Rahul Sharma"
  }
}
```

RESPONSE (ADMIN)

```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor B",
  "routingReason": "Highest priority (3) among eligible vendors",
  "latencyMs": 1190,
  "cost": 0.05,
  "response": {
    "panStatus": "INVALID",
    "nameMatch": true,
    "pan": "ABCDE1234F"
  },
  "failoverHistory": [
    { "vendor": "Vendor A", "status": "timeout", "reason": "Vendor timed out after 2000ms" },
    { "vendor": "Vendor B", "status": "success" }
  ]
}
```

PAN Verification — same request, client role

Identical request, sent with x-role: client. Vendor identity is gone; only what the client actually needs remains:

```
{
  "status": "FAILURE",
  "latencyMs": 3500,
  "cost": 0,
  "response": { "error": "All vendors failed to process the request" }
}
```

OCR

REQUEST

```
POST /route
{
  "capability": "OCR",
  "payload": {
    "documentUrl":
"https://example.com/invoice.jpg",
    "documentType": "INVOICE"
  }
}
```

RESPONSE (ADMIN)

```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor C",
  "routingReason": "Highest priority (4) among
eligible vendors",
  "latencyMs": 1629,
  "cost": 0.2,
  "response": {
    "extractedText": "M/S ACME TRADING CO. |
INVOICE
#4821 | AMOUNT: 12,450.00",
    "confidence": 0.99,
    "fieldsDetected": { "documentType": "INVOICE",
"pageCount": 1 }
  },
  "failoverHistory": [
    { "vendor": "Vendor A", "status": "failure" },
    { "vendor": "Vendor C", "status": "success" }
  ]
}
```

SMS

REQUEST

```
POST /route
{
  "capability": "SMS",
  "payload": {
    "to": "+919876543210",
    "message": "Your OTP is 482913"
  }
}
```

RESPONSE (ADMIN)

```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor C",
  "routingReason": "Highest priority (4) among
eligible vendors",
  "latencyMs": 150,
  "cost": 0.2,
  "response": {
    "messageId": "sms_x50pktgp",
    "deliveryStatus": "DELIVERED",
    "to": "+919876543210"
  }
}
```

Payment Processing

REQUEST

```
POST /route
{
  "capability": "PAYMENT_PROCESSING",
  "payload": {
    "amount": 999.0,
    "currency": "INR",
    "cardLast4": "4242"
  }
}
```

RESPONSE (ADMIN)

```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor B",
  "routingReason": "Highest priority (3) among
eligible vendors",
  "latencyMs": 697,
  "cost": 0.05,
  "response": {
    "transactionId": "txn_pwosd9iv",
    "paymentStatus": "CAPTURED",
    "amountCharged": 999,
    "currency": "INR"
  },
  "failoverHistory": [
    { "vendor": "Vendor C", "status": "timeout" },
    { "vendor": "Vendor B", "status": "success" }
  ]
}
```

Document Validation

REQUEST

```
POST /route
{
  "capability": "DOCUMENT_VALIDATION",
  "payload": {
    "documentType": "CONTRACT",
    "documentUrl":
"https://example.com/contract.pdf"
  }
}
```

RESPONSE (ADMIN)

```
{
  "status": "SUCCESS",
  "vendorUsed": "Vendor C",
  "routingReason": "Highest priority (4) among
eligible vendors",
  "latencyMs": 2179,
  "cost": 0.2,
  "response": {
    "validationStatus": "INVALID",
    "documentType": "CONTRACT",
    "issuesFound": []
  }
}
```

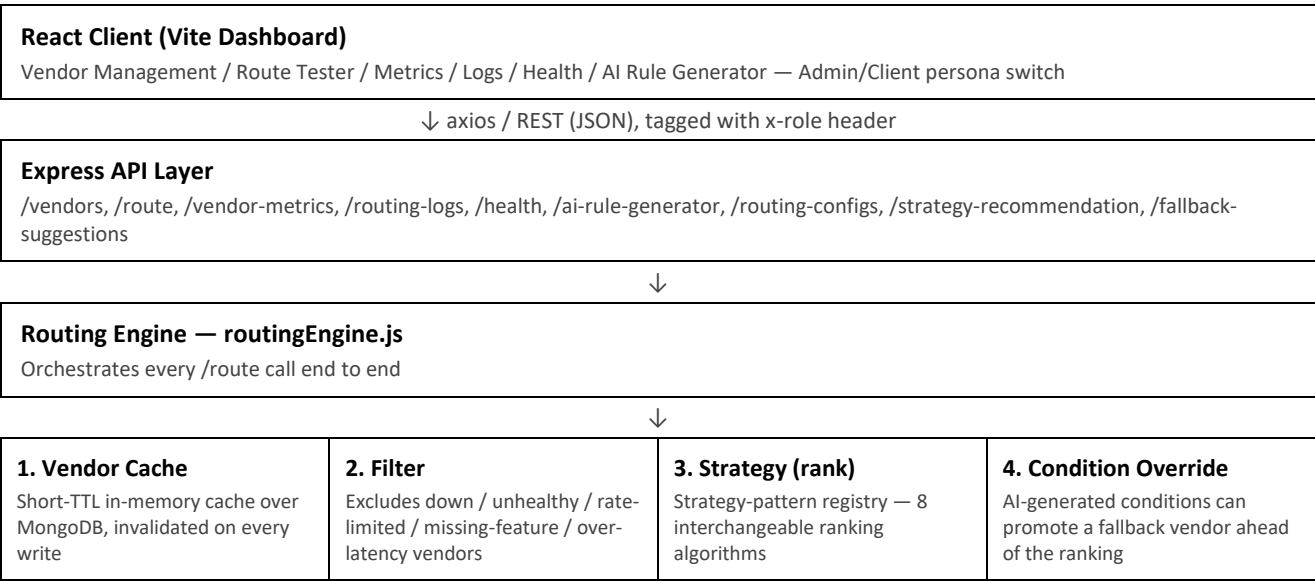
GET /vendor-metrics (aggregate, current)

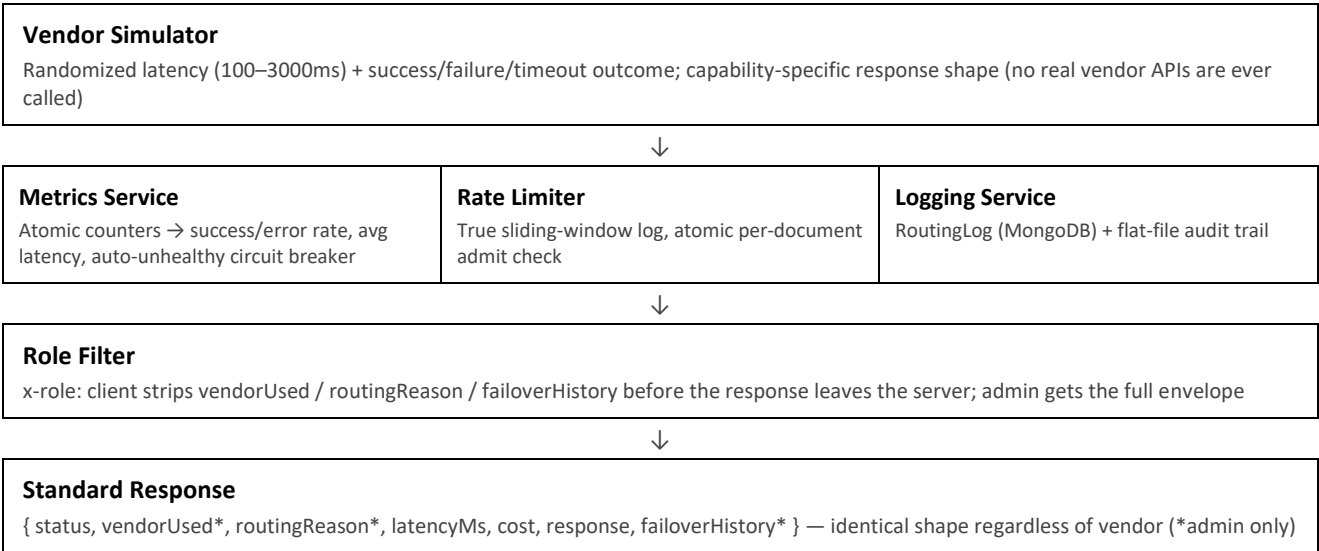
```
{
  "summary": {
    "totalVendors": 5,
    "healthyVendors": 3,
    "totalRequests": 11,
    "successRate": 45.45,
    "averageLatency": 1537
  }
}
```

GET /strategy-recommendation (Agentic AI bonus, current)

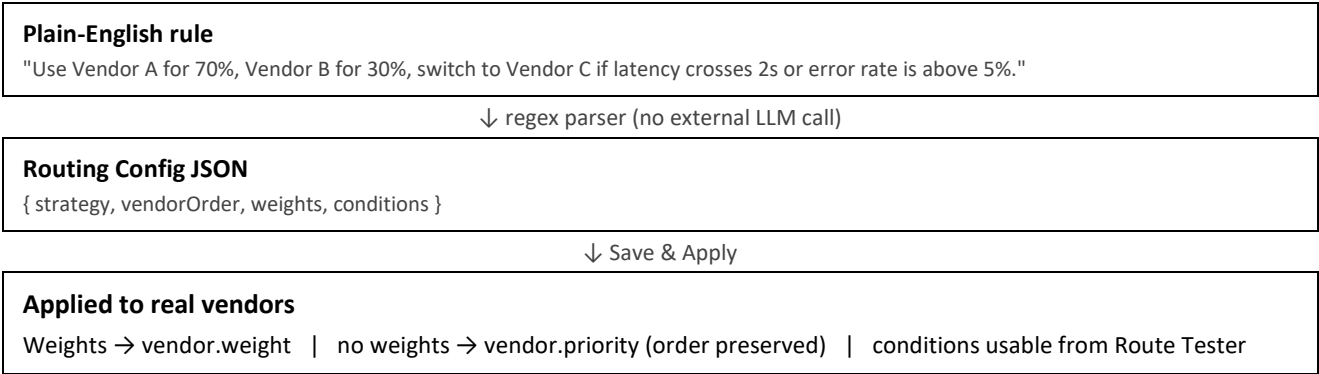
```
{
  "recommendedStrategy": "healthBased",
  "reason": "Vendor reliability varies significantly (health spread 100%, and at
least one vendor has an error rate above 20%). Health-based routing steers
traffic away from the less reliable vendors automatically.",
  "signals": { "vendorsAnalyzed": 3, "totalRequests": 11, "latencySpreadPct": 68,
    "costSpreadPct": 75, "healthSpreadPct": 100 }
}
```

5. Architecture Diagram





Agentic AI layer (bonus)



6. Explanation of Routing Decisions

How one /route call resolves

- Fetch & filter.** All vendors are read (via the short-TTL cache) and each is checked against five exclusion rules: down (isActive: false), unhealthy, rate-limited (sliding window over the last 60s), missing the requested capability, and over the latency threshold. Every excluded vendor is recorded with its exact reason.
- Rank.** The remaining eligible vendors are ordered by whichever strategy was requested (or the sensible default — see below).
- Apply conditions.** If an AI-generated condition applies (e.g. "switch to Vendor C if latency exceeds 2s") and the top-ranked vendor currently breaches it, the named fallback is promoted to the front of the list.
- Attempt with failover.** The engine calls the simulated vendor. On success, it returns immediately. On failure/timeout, it automatically tries the next vendor in the ranked list — continuing until one succeeds or the list is exhausted.
- Record.** Metrics update atomically and a routing log is written.
- Shape by role & respond.** If the caller identified as client (via x-role), vendor-identifying fields are stripped; otherwise the full envelope is returned.

The 8 routing strategies

STRATEGY	RANKING LOGIC
priority	Descending by configured priority value
weighted	Weighted-random sampling without replacement, by weight
roundRobin	Rotates by stable vendor identity — correct even as the eligible set changes shape between calls
lowestLatency	Ascending by currentLatency; untested vendors are ranked after proven ones (not falsely "fastest" at 0ms)
lowestCost	Ascending by costPerRequest
healthBased	Composite score from success rate, an independently-tracked availability metric, and error rate
failover	Strict priority order — the canonical "primary, then failover chain" strategy
featureBased	Prefers vendors supporting the broadest additional feature set as a tiebreak

strategy is optional on /route: if omitted, a requirements.preferLowCost: true hint picks lowestCost, otherwise it defaults to priority.

Why a vendor was (or wasn't) selected

Every admin-role response carries a routingReason string generated per strategy (e.g. "Highest priority (5) among eligible vendors", "Lowest cost per request (\$0.05) among eligible vendors"), plus a full failoverHistory listing every vendor that was considered, whether it was skipped or attempted, and exactly why. A client-role response gets none of this — only status, latencyMs, cost, and response.

Automatic failover triggers

TRIGGER	MECHANISM
Vendor down	Excluded before ranking (isActive: false)
Timeout	Simulated call exceeds the vendor's timeoutMs → next vendor attempted
High latency	Excluded before ranking if currentLatency exceeds the threshold
High error rate	Auto-circuit-breaker flips the vendor unhealthy once its rolling error rate crosses a threshold over enough requests, excluding it from future routing
Rate limit exceeded	Atomic, race-free sliding-window check — excluded before ranking, or the attempt itself is rejected if a burst of concurrent requests would exceed the limit
Unsupported feature	Excluded before ranking if the vendor's supportedFeatures doesn't include the requested capability

7. AI Usage

Full disclosure lives in AI_USAGE.md at the repository root. Summary:

AI used to build this project

Claude Code (Claude Sonnet 5) was used as an AI pair-programmer throughout development — scaffolding, implementation, the test suite, and iterative debugging. A human directed all scope decisions, supplied the assignment brief, caught real defects during manual testing (which prompted fixes), and requested explicit self-audits against the brief at multiple points. Every change was verified against a live, running instance of the app — via curl, the automated test suite, and real browser sessions (screenshots + console-error checks) — rather than accepted on the basis of code generation alone.

AI-branded features inside the product (the bonus)

The AI Rule Generator, strategy recommender, unhealthy-vendor detector, and fallback suggester are all implemented as rule-based logic and statistical heuristics — not calls to an external LLM API, for the same reason vendors are simulated rather than real: deterministic, inspectable, unit-testable behavior for the actual grading criteria, with no dependency on a third-party model's availability or non-determinism. See table in `AI_USAGE.md` for exactly which file implements which bonus feature.